TP N°4 : Architecture Orientée Services (SOA)

Encadrant : Pr. BE ELBAGHAZAOUI

Établissement : ENSA BM

Réalisée par : Sofyan MEZIANE

Année universitaire : 2025-2026

1. Introduction

L'architecture orientée services (SOA - Service-Oriented Architecture) est un paradigme de conception logicielle permettant de structurer une application en services indépendants, modulaires et réutilisables. Ce travail pratique vise à implémenter une application e-commerce basée sur ce paradigme en utilisant le framework Spring Boot.

1.1 Objectifs pédagogiques

- Comprendre les principes fondamentaux de l'architecture SOA
- Créer des microservices indépendants avec Spring Boot
- Implémenter la communication inter-services via REST et OpenFeign
- Gérer une base de données embarquée avec Spring Data JPA
- Mettre en place un service de découverte dynamique (Eureka)

1.2 Prérequis techniques

Outil	Version minimale	Rôle
Java JDK	17+	Environnement d'exécution
Maven	3.6+	Gestionnaire de dépendances
Spring Boot	3.2.x	Framework principal
IDE	IntelliJ / Eclipse / VS Code	Environnement de développement
Postman / cURL	Toute version	Test des APIs REST

2. Architecture du Système

L'application e-commerce développée dans ce TP est composée de quatre microservices distincts, chacun possédant sa propre base de données en mémoire (H2). Cette séparation garantit l'indépendance et la résilience de chaque service.

2.1 Description des microservices

Service	Port	Description
Eureka Server	8761	Service de découverte et registre des microservices
Product Service	8081	Gestion du catalogue de produits
Order Service	8082	Gestion des commandes clients

Service	Port	Description
API Gateway	8080	Point d'entrée unique pour les requêtes clients

2.2 Communication inter-services

La communication entre les services repose sur deux mécanismes complémentaires :

- REST/HTTP : communication synchrone entre services via des appels HTTP standard
- OpenFeign : client déclaratif qui simplifie les appels inter-services (utilisé dans Order Service pour appeler Product Service)
- Eureka : enregistrement automatique et découverte dynamique des instances de services

3. Étape 1 : Serveur Eureka

Le serveur Eureka constitue le cœur du système de découverte de services. Il permet aux microservices de s'enregistrer au démarrage et de se localiser mutuellement dynamiquement.

3.1 Création du projet

1. Accéder à https://start.spring.io/
2. Configurer : Maven, Java, Spring Boot 3.2.x, Artifact: eureka-server
3. Ajouter la dépendance : Eureka Server
4. Télécharger et extraire le projet

3.2 Configuration (application.properties)

```
server.port=8761
spring.application.name=eureka-server
eureka.client.register-with-eureka=false
eureka.client.fetch-registry=false
eureka.client.service-url.defaultZone=http://localhost:8761/eureka/
```

3.3 Classe principale

```
@SpringBootApplication
@EnableEurekaServer
public class EurekaServerApplication {
    public static void main(String[] args) {
        SpringApplication.run(EurekaServerApplication.class, args);
    }
}
```

Démarrer le service avec :

```
mvn spring-boot:run
```

Vérifier le tableau de bord à l'adresse : http://localhost:8761

4. Étape 2 : Product Service

Le Product Service gère l'ensemble du catalogue de produits. Il expose une API REST CRUD complète et s'enregistre automatiquement auprès du serveur Eureka.

4.1 Dépendances Maven

- spring-boot-starter-web
- spring-boot-starter-data-jpa
- h2 (base de données en mémoire)
- spring-cloud-starter-netflix-eureka-client
- lombok

4.2 Modèle de données - Product.java

```
@Entity @Data @NoArgsConstructor @AllArgsConstructor
public class Product {
    @Id @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;
    private String name;
    private String description;
    private Double price;
    private Integer stock;
}
```

4.3 Couches Repository, Service et Controller

L'architecture en couches garantit la séparation des responsabilités :

- ProductRepository : hérite de JpaRepository, accès aux données sans code boilerplate
- ProductService : logique métier (getAllProducts, getById, create, delete)
- ProductController : exposition des endpoints REST sur /api/products

4.4 Endpoints REST exposés

Méthode HTTP	Endpoint	Description
GET	/api/products	Récupérer tous les produits
GET	/api/products/{id}	Récupérer un produit par ID

Méthode HTTP	Endpoint	Description
POST	/api/products	Créer un nouveau produit
DELETE	/api/products/{id}	Supprimer un produit

5. Étape 3 : Order Service

Le Order Service gère les commandes clients. Il communique avec le Product Service via OpenFeign pour récupérer les informations produit lors de la création d'une commande.

5.1 Dépendances supplémentaires

- spring-cloud-starter-openfeign : client HTTP déclaratif pour les appels inter-services

5.2 Modèle Order et statuts

Le modèle Order inclut les champs suivants :

Champ	Type	Description
id	Long	Identifiant unique auto-généré
productId	Long	Référence au produit commandé
productName	String	Nom du produit (dénormalisé)
quantity	Integer	Quantité commandée
totalPrice	Double	Prix total calculé
orderDate	LocalDateTime	Date de la commande
status	String	PENDING / CONFIRMED / SHIPPED / DELIVERED

5.3 Client Feign - ProductClient.java

```
@FeignClient(name = "product-service")
public interface ProductClient {
    @GetMapping("/api/products/{id}")
    ProductDTO getProductById(@PathVariable Long id);
}
```

5.4 Logique métier - création de commande

Lors de la création d'une commande, le service :

5. Récupère les informations du produit via le client Feign
6. Calcule le prix total (prix unitaire x quantité)

7. Enregistre la commande avec le statut PENDING
8. Retourne la commande créée en JSON

6. Tests et Validation

6.1 Ordre de démarrage des services

Les services doivent être démarrés dans l'ordre suivant pour éviter les erreurs de découverte :

9. Eureka Server (port 8761) - registre de services
10. Product Service (port 8081)
11. Order Service (port 8082)

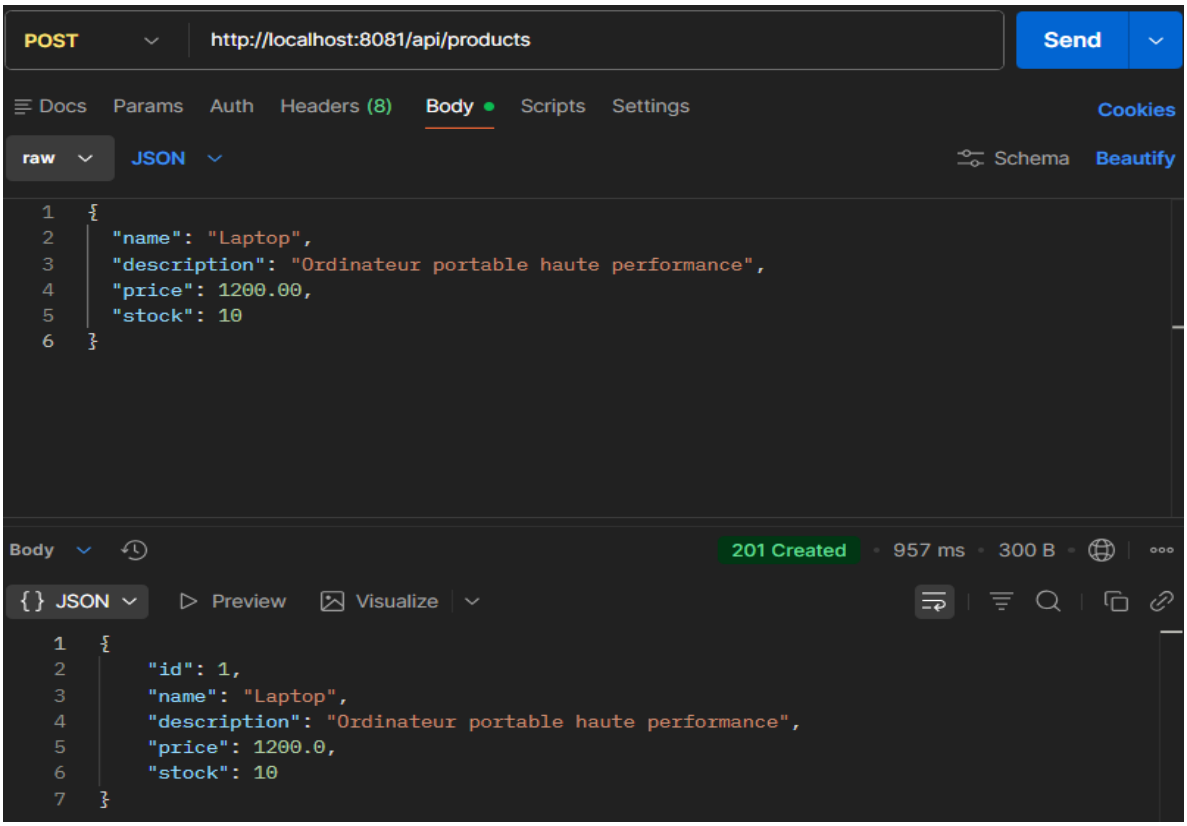
Instances currently registered with Eureka

Application	AMIs	Availability Zones	Status
ORDER-SERVICE	n/a (1)	(1)	UP (1) - 192.168.220.1:order-service:8082
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.220.1:product-service:8081

6.2 Scénarios de test avec Postman

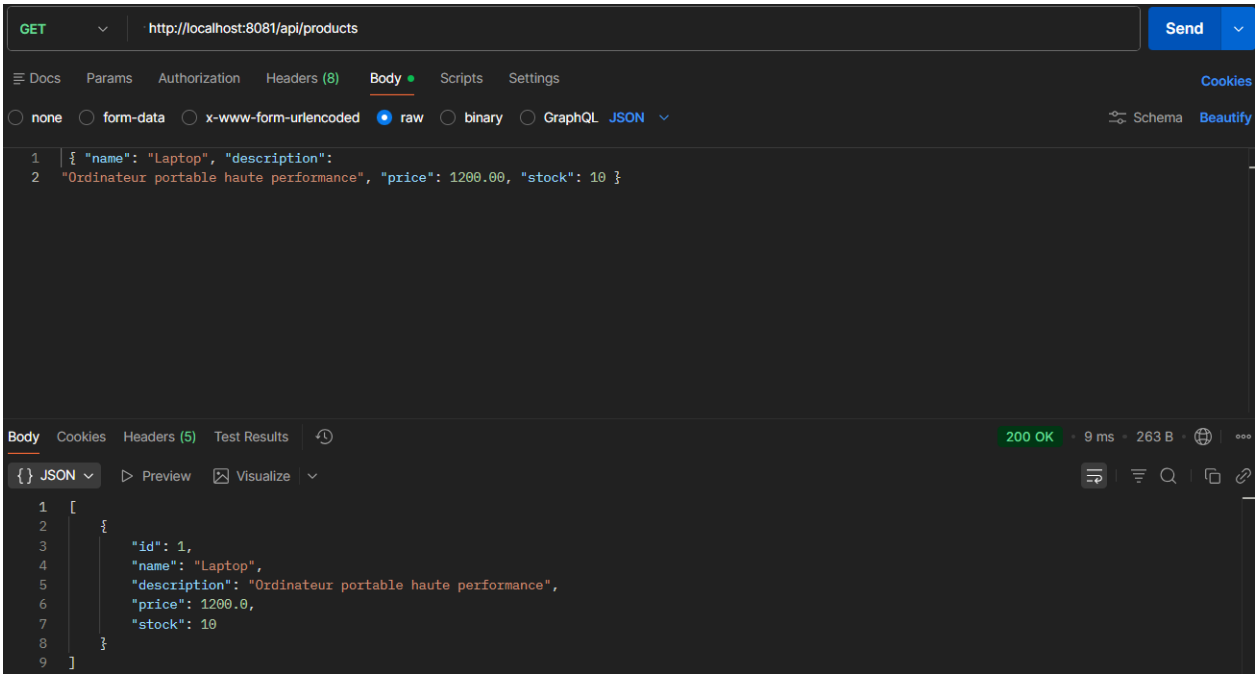
Test 1 :

- Créer un produit :



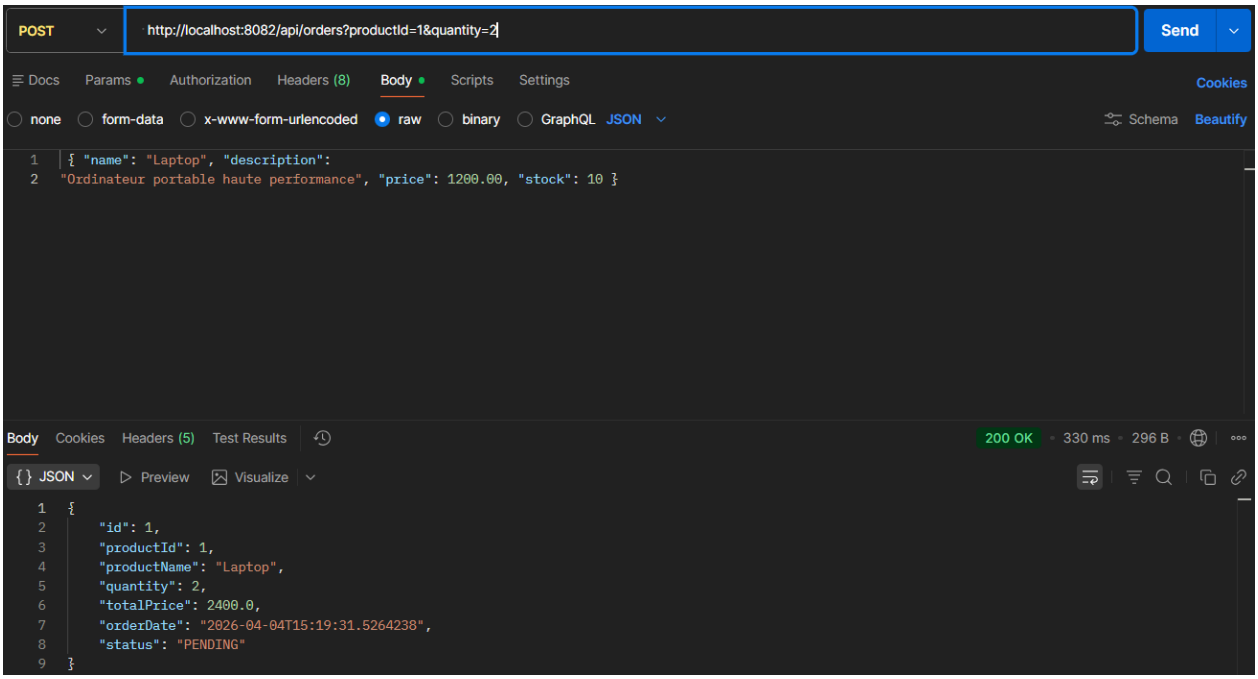
Test 2 :

- lister les produits :



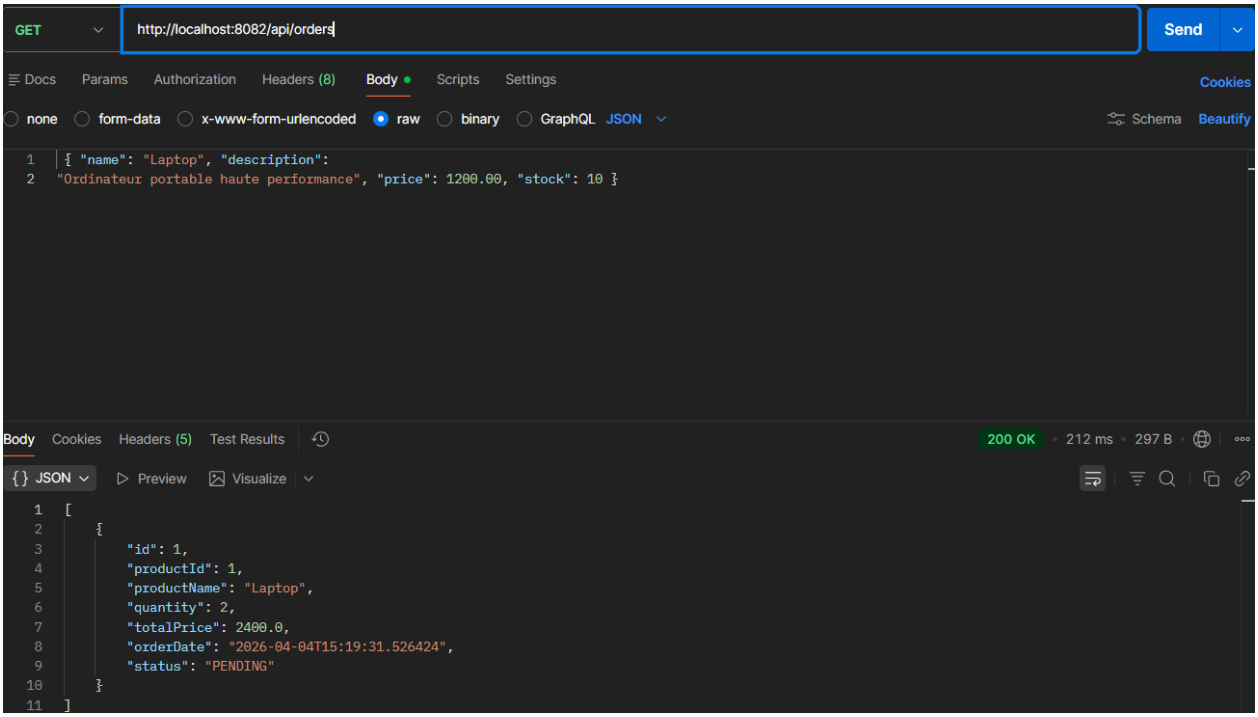
Test 3 :

- **Créer une commande :**



Test 4 :

- Lister les commandes :

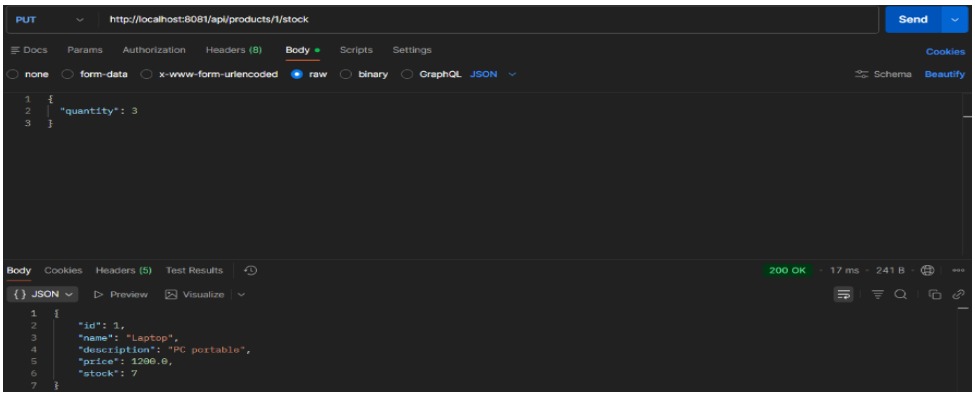


7. Exercices Pratiques (Dans le code source)

7.1 Exercice 1 : Gestion du stock

L'objectif est d'assurer la cohérence des stocks lors des commandes.

- Créer un endpoint PUT /api/products/{id}/stock dans Product Service



- Modifier Order Service pour décrémenter le stock après chaque commande

  Remplacant la methode ‘createOrder’ par :

```
public Order createOrder(Long productId, Integer quantity) {
    ProductDto product = productClient.getProductById(productId);
    productClient.updateStock(productId, new UpdateStockRequest(quantity));
    Order order = new Order();
    order.setProductId(productId);
    order.setProductName(product.getName());
    order.setQuantity(quantity);
    order.setTotalPrice(product.getPrice() * quantity);
    order.setOrderDate(LocalDateTime.now());
    order.setStatus("PENDING");
    return orderRepository.save(order);
}
```

- Gérer les erreurs si le stock est insuffisant :

```
Body  Cookies  Headers (4)  Test Results                    400 Bad Request  17 ms  191 B
Raw  Preview  Debug with AI
1  Stock insuffisant. Disponible: 3, Demandé: 999
```

7.2 Exercice 2 : API Gateway

Créer un service Spring Cloud Gateway centralisant toutes les requêtes entrantes.

- Nouveau projet avec dépendance Spring Cloud Gateway
- Configurer les routes : /products/** vers Product Service, /orders/** vers Order Service
- Port dédié : 8080 :

```
application.properties   ApiGatewayApplication.java
1  server.port=8080
2  spring.application.name=api-gateway
```

- Tester via http://localhost:8080/products et http://localhost:8080/orders

Application	AMIs	Availability Zones	Status
API-GATEWAY	n/a (1)	(1)	UP (1) - 192.168.220.1:api-gateway:8080
ORDER-SERVICE	n/a (1)	(1)	UP (1) - 192.168.220.1:order-service:8082
PRODUCT-SERVICE	n/a (1)	(1)	UP (1) - 192.168.220.1:product-service:8081

7.3 Exercice 3 : Gestion des erreurs

```
exeptions
    ErrorResponse
    GlobalExceptionHandler
    InsufficientStockException
    ProductNotFoundException
    ServiceUnavailableException
```

Améliorer la robustesse du système en gérant les cas d'échec :

- Produit introuvable : retourner HTTP 404 avec message descriptif : ‘Produit introuvable avec l'id’

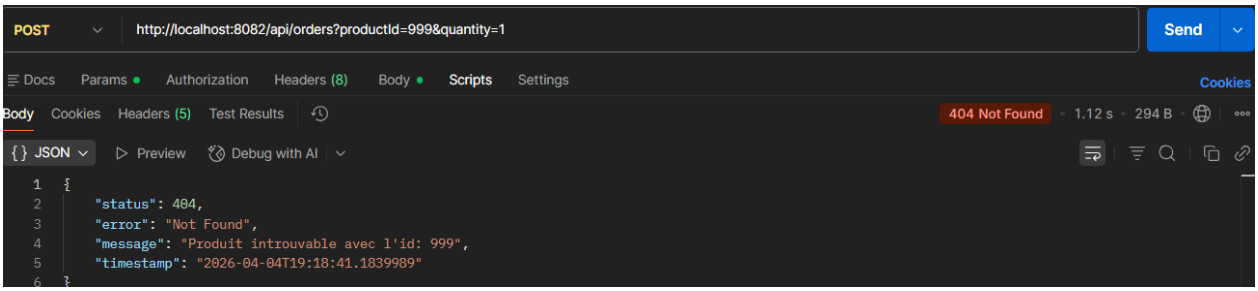
```
package com.ecommerce.order service.exeptions;

public class ProductNotFoundException extends RuntimeException {
    public ProductNotFoundException(Long id) {
        super("Produit introuvable avec l'id: " + id);
    }
}
```

- Gérer le cas où le service Product est indisponible :

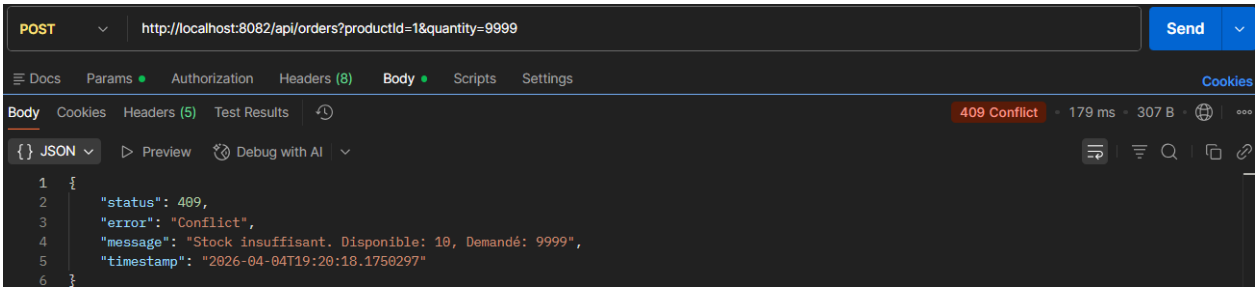
```
package com.ecommerce.order service.exeptions;

public class InsufficientStockException extends RuntimeException {
    public InsufficientStockException(int available, int requested) {
        super("Stock insuffisant. Disponible: " + available
                + ", Demandé: " + requested);
    }
}
```

* Retourner des messages d'erreur appropriés :

-Produit inexistant :



- Stock insuffisant :



8. Conclusion

Ce TP a permis d'explorer en profondeur les concepts fondamentaux de l'architecture orientée services à travers la mise en œuvre d'une application e-commerce complète. L'utilisation de Spring Boot, Eureka et OpenFeign démontre la puissance de l'écosystème Spring Cloud pour la construction de microservices robustes et scalables.

Les exercices pratiques proposés permettent d'approfondir des thématiques critiques en production : gestion des stocks, routage centralisé via API Gateway, et résilience aux pannes. Ces compétences sont directement transférables dans des contextes industriels réels.